Agentic AI Production Readiness Scorecard

Version 1.1 · October 2026 · Nitin Gupta

This scorecard helps an enterprise AI team decide whether an agentic AI system is ready to act in production, and how much authority it has earned. It rests on one idea: enterprises rarely fail at AI because the model is weak. They fail because nobody decided what the system is allowed to do, nobody proved it deserves that authority, and nobody can take that authority back quickly when it stops being deserved.

Capability is not authority. A system can pass every demo and still not be ready to change anything that matters.

How to score

Scope. Score one agentic AI system at a time, as it runs today. Do not score plans, roadmaps, or work in progress. If a control exists only in a design document, it does not exist yet.

Evidence rule. Every score above 0 must be backed by something the team can show in the room: a document, a configuration, a log, a dashboard, a test result, a signed approval. “We do that” without evidence scores 0.

Lowest true anchor. Each criterion has four anchors (0 to 3). Award the highest score where every condition in that anchor is true. If one condition in an anchor is missing, drop to the anchor below. Anchors are cumulative: a 3 requires everything in the 2.

When in doubt, score lower. Disagreements that cannot be settled by evidence in the workshop are scored at the lower value and logged as an action item.

Gate criteria. Six criteria are marked **GATE**. They carry no extra points, but they cap the readiness band regardless of the total (see Readiness Bands).

Scoring rubric

Each criterion score maps to a maturity level.

Score	Level	Meaning
0	1 - Ad hoc	The practice does not exist, or exists only in someone’s head.
1	2 - Developing	The practice exists in part, informally, or without evidence that it is followed.
2	3 - Defined	The practice is written down, complete for this system, owned by a named person, and followed.
3	4 - Managed	The practice is enforced by systems rather than goodwill, measured, and reviewed on a fixed cadence.

A category’s maturity level is its average criterion score, rounded down, mapped through the same table. A category averaging 1.8 is Level 2, Developing.

Category 1: Authorization and Decision Rights

What the system is allowed to do, who decided that, and how that decision changes.

1.1 Written authority boundary · GATE

The system’s permitted actions are written down: every system it can write to, every action type it can take, and numeric limits on those actions.

- **0:** No written list of permitted actions exists.
- **1:** A written description exists, but it has no numeric limits (value, volume, or record count), or it omits at least one system the agent can write to.
- **2:** The written list covers every system the agent can write to, every action type, and numeric limits for each, and the accountable decision owner (1.2) has approved it.
- **3:** As 2, and the limits are enforced in code or platform configuration, and the document is version-controlled with dated changes.

1.2 Named authority owner · GATE

One named individual holds the right to approve, expand, restrict, or revoke this system's authority.

- **0:** No one is named, or the answer depends on who is asked.
- **1:** A team or committee is named, but no single individual.
- **2:** A named individual is recorded as holding that right, and the record states all four powers: approve, expand, restrict, revoke.
- **3:** As 2, and a named backup exists, and both are listed in an enterprise governance register or AI inventory.

1.3 Current authority state recorded

The system's authority is recorded as one of five states for each action type: observe, recommend, act with approval, bounded autonomy, expanded autonomy.

- **0:** No authority state is recorded.
- **1:** One label is recorded for the whole system, regardless of action type.
- **2:** A state is recorded for each action type the system can take.
- **3:** As 2, and states are recorded separately for each market or business unit where they differ, with the date each state last changed.

1.4 Promotion criteria set in advance

The evidence required to move to a higher authority state was written down before the current state was granted.

- **0:** No promotion criteria exist.
- **1:** Criteria are described in words with no measurable thresholds, or were written after the promotion.
- **2:** Measurable thresholds and a minimum volume of evidence were written down before the current authority state was granted.
- **3:** As 2, and the most recent promotion decision is recorded with the actual evidence compared against those thresholds.

1.5 Demotion and revocation triggers · GATE

Specific conditions that reduce or remove the system's authority are written down, with the required response for each.

- **0:** No triggers exist.
- **1:** Triggers are informal ("we would pull it if something went badly wrong") or have no thresholds.
- **2:** Written triggers exist with measurable thresholds, and each trigger names the required response (restrict, pause, or revoke) and who executes it.
- **3:** As 2, and at least one trigger has been exercised in production or in a recorded drill within the last 12 months.

1.6 Enforced approval for high-consequence actions

Actions above a defined consequence threshold require human approval, and the system enforces it.

- **0:** No approval step exists for any action.
- **1:** Approval is required by policy, but the system can execute without it.
- **2:** The system cannot execute defined high-consequence action classes without a recorded approval.
- **3:** As 2, and each approval record captures approver identity, timestamp, and the evidence the approver was shown.

Category 2: Data Governance

What the system can read and write, and under what terms.

2.1 Data source inventory

Every data source the system can read from or write to is listed with an owner and a classification.

- **0:** No list exists.
- **1:** A list exists but is missing sources, owners, or classifications.
- **2:** A complete list exists, with a named owner and a data classification for every source.
- **3:** As 2, and the list is reconciled against actual access (from logs or access records) at least quarterly.

2.2 Least-privilege access

The system's credentials grant only the access it needs.

- **0:** The system uses shared, human, or administrator credentials.
- **1:** The system has dedicated credentials, but they grant access beyond the sources in 2.1 or do not separate read from write.
- **2:** Credentials are scoped to the sources in 2.1, with read and write permissions separated.
- **3:** As 2, and access is reviewed at least quarterly, with unused permissions removed and the removal recorded.

2.3 Sensitive data controls

Personal, regulated, or confidential data is protected by technical controls, not only policy.

- **0:** No rules exist for sensitive data in this system.
- **1:** A policy exists, but no technical control enforces it.
- **2:** Technical controls (masking, filtering, or redaction) cover defined sensitive fields, and a test shows they work.
- **3:** As 2, and those tests run automatically before every release, and any failures are tracked as incidents.

2.4 Provider data terms confirmed

For every model and tool provider in the system's path, the team knows in writing whether the provider trains on, retains, or relocates enterprise data.

- **0:** Unknown for at least one provider.
- **1:** Based on marketing material or verbal assurance for at least one provider.
- **2:** Confirmed in a contract or data processing agreement for every provider in the path.
- **3:** As 2, and contracts require advance notice of changes to these terms, and the terms were reviewed within the last 12 months.

2.5 Data residency and cross-border flows

Where data travels is mapped for every market the system operates in.

- **0:** Data flows are not mapped.
- **1:** Mapped for the primary model only.
- **2:** Mapped for every component (model, retrieval store, logs, tools) in every market where the system operates.
- **3:** As 2, and legal or privacy has signed off each market's data flows, with a dated record.

Category 3: Evaluation and Testing

How the team knows the system is good enough, before and after launch.

3.1 Enterprise-owned evaluation set

The team has its own test cases, built from its own tasks, not the vendor's demo set.

- **0:** No evaluation set exists; quality is judged from demos or anecdotes.
- **1:** An evaluation set exists, but it was supplied by the vendor, or it does not cover every action type.
- **2:** The enterprise owns an evaluation set with at least 20 cases per action type, drawn from real or realistic tasks, including documented edge cases and expected outcomes.
- **3:** As 2, and the set is refreshed with sampled production cases at least quarterly.

3.2 Pass thresholds set before testing

Quality thresholds were written down before results were seen.

- **0:** No thresholds exist.
- **1:** Thresholds exist but were set after seeing results, or are not written down.
- **2:** Written thresholds exist for each action type, set before testing and approved by the authority owner (1.2).
- **3:** As 2, and thresholds are stricter for irreversible or high-consequence action types, with the reasoning documented.

3.3 Regression testing on every change

Any change to the model version, prompts, tools, retrieval, or configuration triggers a re-run of the evaluation set.

- **0:** No regression testing happens.
- **1:** Testing happens for some changes, manually, at the team's discretion.
- **2:** Every change requires a regression run before release, and results are recorded.
- **3:** As 2, and the run is automated in the release pipeline, and a failed run blocks the release.

3.4 Adversarial and safety testing

The system is tested against misuse, not just normal use.

- **0:** No adversarial testing has been done.
- **1:** Informal attempts only, with no documented test cases.
- **2:** A documented test suite covers prompt injection, attempts to exceed the authority boundary in 1.1, and data leakage, and it was run before launch.
- **3:** As 2, and it runs on every release, and someone outside the build team has tested the system within the last 12 months.

3.5 Production quality sampling

Live outputs are reviewed against the same standard used before launch.

- **0:** No production review happens.
- **1:** Occasional review with no defined sample size or cadence.
- **2:** A defined sample is reviewed on a defined cadence and scored with the pre-launch rubric.
- **3:** As 2, and results are trended over time and presented at the authority review (7.5).

Category 4: Observability and Audit

Whether the enterprise can prove what the system did, and why.

4.1 Complete action trace · GATE

Every state-changing action is logged with: the trigger, the agent identity, the model and version, a reference to the inputs and retrieved context, every tool call, any approval, and the outcome.

- **0:** No action-level logging exists.
- **1:** Some of these fields are logged, or logging covers only some actions.
- **2:** All listed fields are captured for every state-changing action.
- **3:** As 2, and logs are tamper-evident or immutable, and retention meets the enterprise's written policy.

4.2 Reconstruction drill

The team can reconstruct a specific past action end to end.

- **0:** Never attempted, or not possible.
- **1:** Possible, but it took engineers more than one business day.
- **2:** Done within one business day, and demonstrated within the last 6 months.
- **3:** Done within one hour using standard tooling, by someone outside the build team.

4.3 Quality and drift alerting

The system is monitored for quality, not only uptime.

- **0:** Only availability and latency are monitored.
- **1:** Quality metrics exist but nobody is alerted when they cross a threshold.
- **2:** Quality metrics have thresholds, and alerts go to a named owner.
- **3:** As 2, and input drift (change in the data the system sees) is also monitored, and alert routing has been tested.

4.4 Human oversight metrics

The team measures how human reviewers behave, not just how the system behaves.

- **0:** Approval rates, override rates, and review times are not tracked.
- **1:** Tracked, but not reviewed.
- **2:** Reviewed on a defined cadence by a named owner.
- **3:** As 2, and defined thresholds (for example, an approval rate above a set level, or review times below a set level) trigger a check on whether reviewers are still genuinely reviewing.

4.5 Cost per completed action

The team knows what one acceptable, completed action actually costs.

- **0:** Cost is not tracked for this system.
- **1:** Only cost per model call or per token is tracked.
- **2:** Fully loaded cost per accepted, completed action is tracked, including retries, human review time, and rework.
- **3:** As 2, and it is tracked against a written cost ceiling, with an alert when exceeded.

Category 5: Safety and Guardrails

Whether limits hold when the model does something unexpected.

5.1 Tested kill switch · GATE

Named people can stop all state-changing actions quickly, without a code deployment.

- **0:** No kill switch exists, or stopping the system requires a deployment.
- **1:** A kill switch exists, but it has never been tested.
- **2:** It was tested within the last 6 months, it stopped all state-changing actions within 15 minutes, and the people allowed to trigger it are named.
- **3:** As 2, and it was tested within the last 90 days, and it can be scoped (one action type, one market) as well as all-off.

5.2 Limits enforced outside the model · GATE

The authority boundary in 1.1 is enforced by systems the model cannot override.

- **0:** Limits rely only on prompt instructions.
- **1:** Some limits are enforced outside the model; others rely on the prompt.
- **2:** Every limit in 1.1 is enforced outside the model, at the tool layer, a policy engine, or the target system.
- **3:** As 2, and every attempted violation is logged and alerted to a named owner.

5.3 Reversibility classified

Each action type is classified as reversible or irreversible, and irreversible actions are treated differently.

- **0:** Not assessed.
- **1:** Assessed informally, not written down.
- **2:** Every action type is classified in writing, and irreversible action types require approval (1.6) or have a documented compensating action.
- **3:** As 2, and the rollback or compensating procedure has been tested.

5.4 Blast-radius caps

Business-level caps limit how much damage a fault can do before someone notices.

- **0:** No caps exist.
- **1:** Only infrastructure limits (rate limits, timeouts) exist.
- **2:** Business-level caps are enforced, such as records changed per hour or monetary value per day.
- **3:** As 2, and caps are formally reviewed every time the system's authority state changes.

5.5 Delegation controls

Authority cannot grow by passing through other agents or tools.

- **0:** The agent can call other agents or tools without restriction.
- **1:** Delegation paths are documented but not enforced.
- **2:** Every delegation path is listed, and no path grants permissions beyond the authority boundary in 1.1.
- **3:** As 2, and the composed end-to-end behavior is covered by the evaluation set. A single agent that is technically prevented from calling other agents scores 3.

Category 6: Reliability and Operations

Whether the system can be run, recovered, and changed safely.

6.1 AI-specific incident playbook

The team has a playbook for quality failures, not only outages.

- **0:** No playbook exists.
- **1:** Only a general IT incident playbook applies.
- **2:** An AI-specific playbook covers quality degradation as well as outage, with severity levels, a named owner, and steps to restrict or reset authority.
- **3:** As 2, and the playbook has been exercised within the last 12 months.

6.2 Fallback path

The business keeps working when the system is paused.

- **0:** No fallback; the process stops.
- **1:** A manual workaround exists but is not documented or staffed.
- **2:** A documented, staffed fallback exists and has been tested.
- **3:** As 2, and the switch to fallback happens automatically on defined triggers, and that switch has been tested.

6.3 Model and vendor change control

Changes from the model or platform provider do not reach production untested.

- **0:** Provider changes are discovered after they affect behavior.
- **1:** The team subscribes to provider notices, but versions are not pinned.
- **2:** Model and platform versions are pinned, and upgrades pass regression testing (3.3) before rollout.
- **3:** As 2, and an alternative model or provider has been run against the evaluation set within the last 12 months.

6.4 Downstream system dependencies

Every system the agent writes to has defined failure behavior.

- **0:** Dependencies are not listed.
- **1:** Dependencies are listed, with no defined failure behavior.
- **2:** Each dependency has defined timeout, retry, and duplicate-action behavior, and it has been tested.
- **3:** As 2, and dependency health is monitored with alerts.

6.5 Agreed service levels

The business owner and the technical owner agree on what "working" means.

- **0:** No service levels exist.
- **1:** Availability targets only.
- **2:** Availability, latency, and quality targets are agreed in writing with the business owner.
- **3:** As 2, and performance is reported monthly, and any breach triggers a recorded review.

Category 7: Ownership and Accountability

Who answers for this system, and how that answer stays current.

7.1 Business owner

A named person owns the business outcome and its economics.

- **0:** No business owner exists.
- **1:** A team is named, but no individual.
- **2:** A named individual owns the outcome, with a written success metric.
- **3:** As 2, and the metric is reported on a defined cadence and reviewed with that owner.

7.2 Technical owner

A named person owns operating and recovering the system.

- **0:** No technical owner exists.
- **1:** A team is named, but no individual.
- **2:** A named individual owns operations, with on-call coverage defined.
- **3:** As 2, and the on-call process has been tested, and runbooks were updated within the last 90 days.

7.3 Risk owner and written acceptance

A named person has accepted the residual risk of the system's current authority, in writing.

- **0:** No risk owner exists.
- **1:** Risk was discussed, but no one signed an acceptance.
- **2:** A named risk owner signed a written acceptance of residual risk for the current authority state.
- **3:** As 2, and acceptance is re-signed every time the authority state changes.

7.4 Agent identity and inventory

The system is registered as a distinct, identifiable actor.

- **0:** The system is not in any inventory.
- **1:** Listed in an informal register, without a unique identity.
- **2:** Registered with a unique non-human identity, a named owner, and its current authority state.
- **3:** As 2, and the inventory is reconciled against agents actually running in production at least quarterly.

7.5 Scheduled authority review

Authority is reviewed on a schedule, for demotion as well as promotion.

- **0:** No review happens after launch.
- **1:** Reviewed at launch only, or only when something goes wrong.
- **2:** Reviewed on a fixed cadence (at least quarterly for bounded or expanded autonomy), and the review considers both promotion and demotion.
- **3:** As 2, and each review records a decision (expand, maintain, constrain, pause, redesign, or exit) with conditions for the next review.

7.6 Exit criteria

The business conditions under which the initiative is redesigned, paused, or stopped are written down.

- **0:** No exit criteria exist.
- **1:** Discussed informally only.
- **2:** Written economic and risk conditions define when the initiative is redesigned, paused, or stopped.
- **3:** As 2, and the criteria were checked against current evidence at the most recent authority review.

Score summary

Category	Criteria	Max	Score	Level
1. Authorization and Decision Rights	6	18		
2. Data Governance	5	15		
3. Evaluation and Testing	5	15		
4. Observability and Audit	5	15		
5. Safety and Guardrails	5	15		
6. Reliability and Operations	5	15		
7. Ownership and Accountability	6	18		
Total	**37**	**111**		

Gate criteria: 1.1 ___ · 1.2 ___ · 1.5 ___ · 4.1 ___ · 5.1 ___ · 5.2 ___

Readiness bands

Apply the gate rules first. The total score cannot lift a system above the band its gates allow. Expanded-ready overlaps the top of Production-ready on purpose: it is the same score range with a stricter gate condition. A spreadsheet edition that applies these rules automatically is in scorecard-workbook.xlsx.

Band	Total score	Gate condition	Maximum authority state
Not ready	0 to 55	Or any gate scored 0	Observe only
Pilot-ready	56 to 83	Every gate at 1 or higher	Recommend, or act with approval in a limited scope
Production-ready	84 to 111	Every gate at 2 or higher, and no criterion at 0	Bounded autonomy within the written limits in 1.1
Expanded-ready	95 to 111	Every gate at 3, and no criterion at 0	Expanded autonomy, with a 60-day sunset on every grant

Not ready. The system may be impressive in a demo, but the enterprise cannot yet say what it is allowed to do, prove what it did, or stop it quickly. Run it in observe mode only, where it logs what it would have done without acting. Next steps: fix every gate scored 0 before anything else, then name the authority owner (1.2), the business owner (7.1), and the risk owner (7.3), because most other gaps cannot close without them. Re-score in 4 to 8 weeks.

Pilot-ready. The basics of control exist, but the evidence is not yet strong enough to let the system act on its own. Run a bounded pilot where the system recommends or acts only with approval, in a limited scope (one market, one action type, capped volume). Next steps: use the pilot to build the evidence the scorecard is missing, especially production sampling (3.5), oversight metrics (4.4), and cost per completed action (4.5). Lift every gate to 2. Re-score before any request to expand scope.

Production-ready. The enterprise can say what the system may do, show what it did, measure whether it is still good enough, and take its authority back quickly. Bounded autonomy within the written limits is defensible. Next steps: hold the scheduled authority review (7.5) and treat this score as perishable, because a model update, a new market, or a new action type can drop it. Re-score at least quarterly and at every authority change.

Expanded-ready. A stricter condition inside the top band, not a separate score range. Bounded autonomy has run long enough to produce sustained evidence, every gate is enforced by systems rather than goodwill, and nothing scores 0. Expanded autonomy is defensible for the action types where that evidence exists. Next steps: record each grant on an Authority Change Record with a 60-day sunset, re-score before every renewal, and keep the scoped kill switch (5.1 at 3) tested on the 90-day cycle. A system that meets this band for one action type has not met it for any other.

One more rule. A high total does not cover a gap in a gate. A system scoring 100 with an untested kill switch is not production-ready. Strong evidence in five places does not make up for missing evidence in the sixth.

What this scorecard is not

It is not a certification, a legal opinion, or a substitute for regulatory compliance work. It is a structured way for a team to have an honest conversation about readiness, using evidence instead of confidence. It draws on concepts from the NIST AI Risk Management Framework, ISO/IEC 42001, and the EU AI Act's human-oversight provisions, and it is not affiliated with or endorsed by any of them.

Based on the authority framework in *The Enterprise AI Decision System: How AI Leaders Turn Capability into Authorized Enterprise Action* by Nitin Gupta.

Latest version, full documentation, and workshop guide: github.com/ngupta1137/enterprise-ai-decision-toolkit/tree/main/scorecard. Part of the Enterprise AI Decision Toolkit. Licensed under MIT.